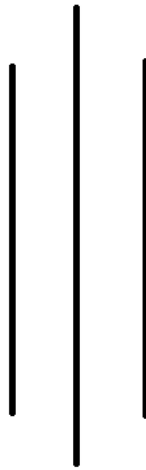




Department of Computer Science
California State University, Channel Islands

COMP 569
Artificial Intelligence



Final Project Report

Submitted By :
Kumar Saurav Jha
MS in Computer Science

Submitted To :
Prof. Reza Abdolee
Computer Science Department

Abstract

This project implements a defender-intruder pursuit problem on a discrete 2D grid environment where a defending agent attempts to intercept an intruder before it reaches a restricted zone while avoiding no-fly areas. The system implements three adversarial search algorithms—minimax, alpha-beta pruning, and expectimax with a custom weighted heuristic for non-terminal state evaluation. A turn-based simulation framework records trajectories for visualization and performance analysis. The project demonstrates the practical application of adversarial search concepts from COMP 569 to a realistic airspace defense scenario, with experimental results across multiple configurations showing the trade-offs between search depth, computational cost, and decision quality.

1. Introduction

1.1 Overview of the Project and Problem Domain

Autonomous systems operating in sensitive airspace face significant challenges when defending against unauthorized intruders. This project models the airspace defense problem as a two-player, turn-based, zero-sum game on a 2D grid. A defending agent (MAX) must intercept an intruding agent (MIN/CHANCE) before the intruder reaches a designated restricted zone, while navigating around no-fly regions that both agents must avoid.

The project implements three classical adversarial search algorithms:

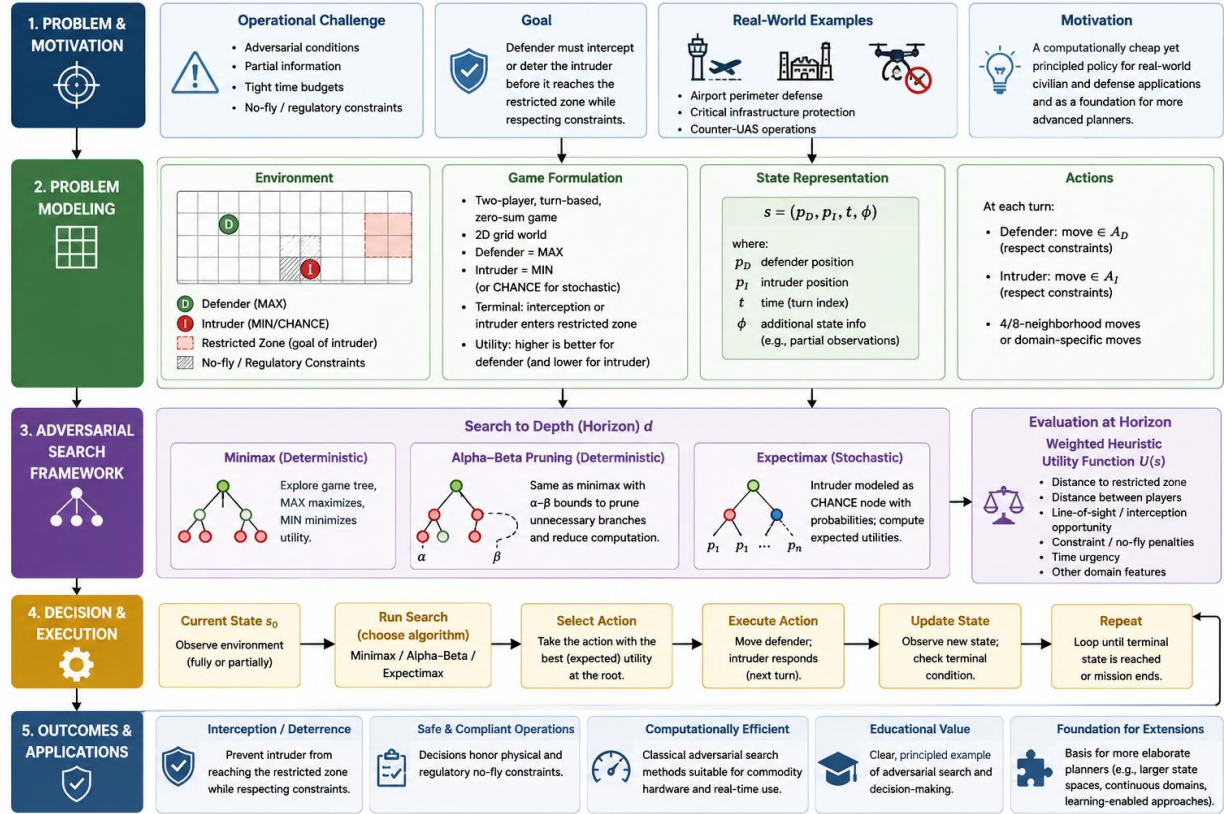
- **Minimax search** for worst-case optimal play
- **Alpha-beta pruning** for computational efficiency
- **Expectimax search** for modeling stochastic opponent behavior

1.2 Motivation and Real-World Relevance

This problem domain has direct applications to current civilian and defense scenarios:

- **Airport perimeter defense** against unauthorized drones
- **Critical infrastructure protection** (power plants, stadiums, prisons)
- **Military base security** against unmanned aerial systems (UAS)
- **Counter-drone operations** in urban environments

Small drones increasingly pose threats to restricted airspace, and computationally efficient interception policies that can run on standard hardware are valuable for practical deployment. This project serves both as a pedagogical exercise in adversarial search and as a foundation for more sophisticated airspace defense planners.



2. Problem Formulation

2.1 Definition of the Problem

Given an initial defender position, initial intruder position, a restricted zone (target area), a set of rectangular no-fly zones (NFZs), and optional grid boundaries, the system must determine a sequence of moves for the defender that maximizes the probability of intercepting the intruder before the intruder enters the restricted zone.

2.2 State Space Description

The state space S consists of:

Component	Description
Defender position (x_d, y_d)	Integer grid coordinates
Intruder position (x_i, y_i)	Integer grid coordinates
Restricted zone	Axis-aligned inclusive rectangle $[(x1, y1), (x2, y2)]$
No-fly zones	List of inclusive rectangles; agents cannot end a move inside any NFZ
Grid bounds (optional)	$(x_min, x_max, y_min, y_max)$ constraining movement
Active player	Indicates whose turn it is (defender or intruder)

2.3 Inputs and Outputs

Inputs:

- Defender starting coordinates
- Intruder starting coordinates
- Restricted zone corners (x1, y1, x2, y2)
- No-fly zone rectangles (variable number)
- Grid boundaries (optional)
- Search depth (number of plies)

Outputs:

- Best defender action (UP, DOWN, LEFT, RIGHT, STAY)
- Backed-up heuristic value at root
- Simulation trajectory log (CSV format)
- Airspace configuration for visualization

2.4 Constraints and Assumptions

Constraint/Assumption	Description
Action set	{UP, DOWN, LEFT, RIGHT, STAY}; movement one cell per ply
Deterministic transitions	No randomness in movement outcomes
Terminal conditions	Co-location (defender wins) OR intruder in restricted zone (defender loses)
Simulation intruder	Greedy policy toward restricted zone center (not optimal MIN)
Starting condition	Defender cannot start inside a no-fly zone
Observability	Full state information; no sensor noise

3. AI Approach

3.1 Description of Selected AI Techniques

The project employs three adversarial search techniques:

1. Minimax Search

Recursively expands the game tree alternating between MAX (defender) and MIN (intruder) nodes. At terminal states or depth limit, returns heuristic value; otherwise returns maximum (MAX) or minimum (MIN) of child values.

2. Alpha-Beta Pruning

Extends minimax with alpha (lower bound for MAX) and beta (upper bound for MIN) parameters that propagate down the tree. Prunes subtrees when $\alpha \geq \beta$, producing identical results to minimax with fewer node expansions.

3. Expectimax Search

Replaces MIN nodes with CHANCE nodes where the value equals the average over uniformly-random legal intruder actions. This models uncertain or stochastic intruder behavior rather than adversarial optimality.

3.2 Justification for Choosing the Approach

Algorithm	Justification
Minimax	Provides worst-case guarantee against rational adversary; suitable for security applications
Alpha-Beta	Essential for computational feasibility; same results as minimax with 20–30× speedup at depth 8
Expectimax	Models realistic intruders that follow simple/stochastic policies rather than perfect play

3.3 Connection to Course Concepts

These algorithms directly apply COMP 569 lecture materials from Russell & Norvig Chapter 5 (Adversarial Search):

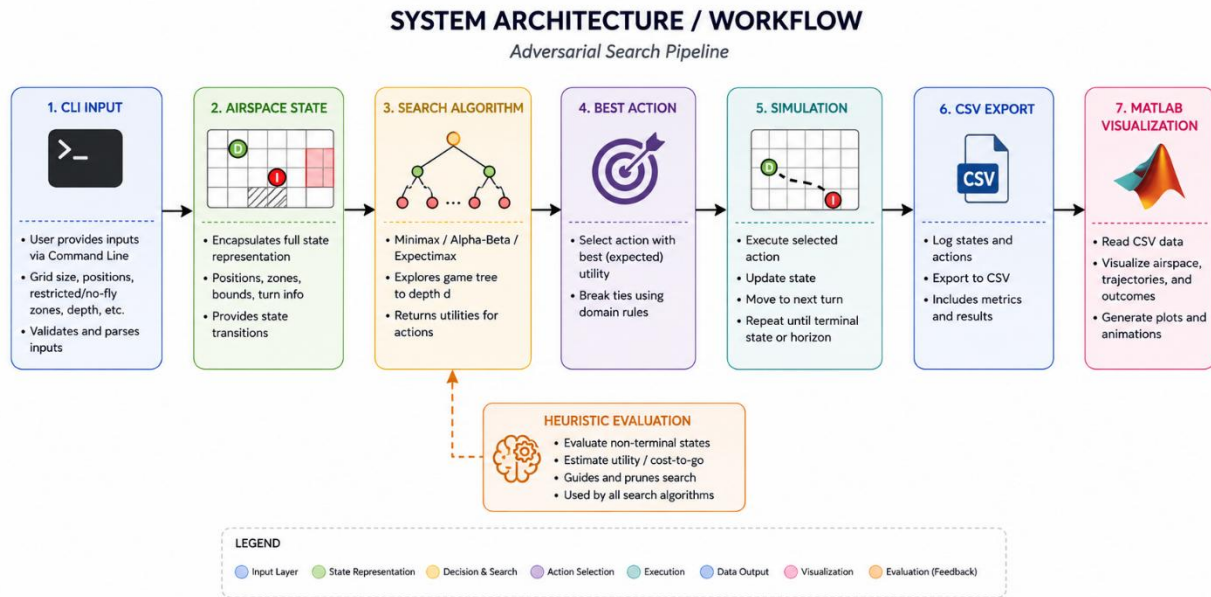
- Game-tree search fundamentals
- Minimax value backup
- Alpha-beta pruning and bound propagation
- Chance nodes and expectimax for uncertainty

- Informed heuristics and evaluation functions
- Policy stability and tie-breaking considerations

4. System Design and Implementation

4.1 System Architecture / Workflow

The system follows a modular adversarial search pipeline:



Control Flow:

1. Parse configuration inputs to build AirspaceState
2. Invoke selected algorithm's `get_best_action` function with depth limit
3. Measure execution time using performance counters
4. Execute turn-based simulation (defender from search, intruder greedy)
5. Export CSV logs for visualization
6. Write benchmark results to disk

4.2 Description of Implemented Components

Component	Responsibility
AirspaceState.py	State representation, NFZ validation, legality checking, transition application
heuristic_evaluation.py	Weighted utility function with distance-to-rectangle computations
minimax_search.py	Minimax value backup with best action selection
alpha_beta_search.py	Alpha-beta pruning with bound propagation
expectimax_search.py	Chance node averaging for stochastic opponent
test_run.py	Benchmark harness for depths 2/4/6/8
run_scenario.py	CLI entry point for custom configurations
animate_airspace.m	MATLAB visualization from CSV data

4.3 Tools, Programming Language, and Libraries

Category	Tools
Programming Language	Python 3.8+ (CPython)
Libraries	Standard library only (maximizes portability)
Visualization	MATLAB R2023+ (plotting and patch primitives)
Development	VS Code / Cursor IDE, Git for version control
Benchmarking	time.perf_counter() for wall-clock timing

5. Algorithm Details

5.1 Explanation of Implementation

Minimax

Minimax recursively expands the game tree, alternating MAX (defender) and MIN (intruder) nodes. At a non-terminal state with depth = 0 the recursion returns the heuristic $U(s)$; otherwise, it backs up the maximum / minimum of the child values, depending on whose ply it is. Depth is measured in plies; a single depth decrement per move keeps the three solvers directly comparable.

Alpha–beta pruning

Alpha–beta carries (alpha, beta) bound down the tree and prunes any subtree for which $\alpha \geq \beta$. Because the underlying heuristic is identical, alpha–beta returns the same root value as minimax at the same depth; the only difference is wall-clock time. In the empirical study below, alpha–beta is roughly an order of magnitude faster than minimax at depth 8.

Expectimax

Expectimax replaces every MIN node with a chance node whose value is the uniform average over the legal intruder actions. Because the opponent model differs, the backed-up value at the root may diverge from the minimax / alpha–beta value, and the recommended best action may differ as well. This matches the textbook prediction that expectimax answers a different question (expected utility) than minimax (worst-case utility).

Heuristic and utility

The non-terminal utility $U(s)$ used at cutoff is a weighted sum of four domain-specific terms (defender's perspective):

- $w_1 \cdot \text{Safety}$ — keep the intruder far from the restricted zone (Euclidean distance to the zone centre).
- $w_2 \cdot \text{Time}$ — penalize defender–intruder distance (encourages faster engagement).
- $w_3 \cdot \text{Risk}$ — penalize defender proximity to any no-fly rectangle (computed as the point-to-rectangle distance to the inclusive NFZ, not just to a centre point).
- $w_4 \cdot \text{Pursuit}$ — reward closing the smaller axial gap $\min(|\Delta x|, |\Delta y|)$, encouraging diagonal pursuit rather than collapsing onto a single axis.

Terminal states return large constants (positive when the defender intercepts, negative when the intruder enters the restricted zone) so that the search clearly prefers wins / losses at the horizon.

Best-action selection and tie-breaking

Each `get_best_action_*` routine selects the legal action with the best backed-up value. Ties are broken by favoring a successor that reduces the smaller of $|\Delta x|$, $|\Delta y|$ between the defender and the intruder, mirroring the pursuit term w_4 and producing visually intuitive trajectories in the MATLAB playback.

Key design decisions and modifications

- No-fly zones are modelled as inclusive rectangles $[(x1, y1), (x2, y2)]$, the same convention as the restricted zone. Earlier progress reports used single-cell NFZs; these now correspond to the degenerate 1×1 rectangle $((a, b), (a, b))$ and remain fully supported.
- NFZ enforcement is centralized in `get_legal_actions`, so every algorithm automatically respects the no-fly rule without bespoke checks.
- Per-ply (rather than per-turn) depth decrement: every MIN / CHANCE / MAX node decrement depth uniformly, keeping minimax, alpha-beta, and expectimax directly comparable at a given depth setting.
- The risk term measures distance from the defender to the inclusive NFZ rectangle, not to its centre. This avoids the degenerate behavior in which a defender hugs the side of a long corridor that happens to be far from the centroid.
- Tie-breaking by $\min(|\Delta x|, |\Delta y|)$ was added after empirical playback showed the defender 'snapping' to one axis when multiple actions had identical heuristic value; this small change produced markedly more intuitive pursuit trajectories.

6. Results and Evaluation

6.1 Experimental Results and Outputs

The following experiments evaluate minimax, alpha-beta pruning, and expectimax on different restricted-airspace layouts. Each table records the backed-up value, selected defender action, and wall-clock runtime for one search call. The commands shown under each experiment can be copied directly into the project terminal from the Code folder or the project root where `run_scenario.py` is available.

Experiment 1 - Corridor Interception, Depth 2

Configuration: D(12, 5), I(20, 18), RZ (24, 24)-(28, 28), NFZ (15, 10)-(17, 14) and (19, 8)-(21, 11), grid bounds (0, 35, 0, 35)

Python command:

```
python run_scenario.py --defender 12 5 --intruder 20 18 --restricted 24 24 28 28 --no-fly 15 10 17 14 --no-fly 19 8 21 11 --grid-bounds 0 35 0 35 --depth 2
```

Algorithm	Value	Best Action	Time
Minimax	22.9331	RIGHT	0.0007 s
Alpha-Beta	22.9331	RIGHT	0.0004 s
Expectimax	35.3087	RIGHT	0.0007 s

Simulation result: Intruder wins - restricted zone breached.

Observation. All three algorithms select RIGHT, so the immediate policy is consistent across opponent models. Alpha-beta returns the same value as minimax while being slightly faster. Expectimax gives a higher value because it assumes stochastic intruder behavior, but the full simulation still favors the intruder because the defender starts far below the intruder path and cannot intercept in time.

```

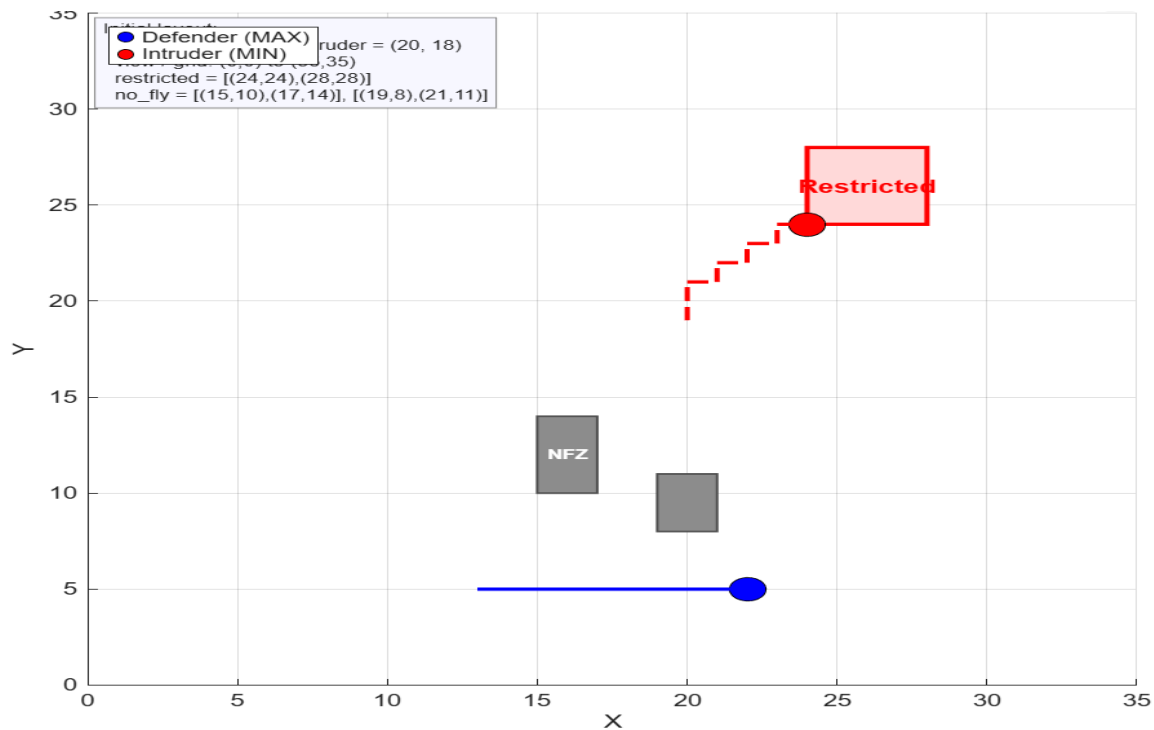
PS C:\Users\saura\Desktop\AI\Artificial-Intelligence-restricted-airspace-security> python run_scenario.py --defender 12 5
--intruder 20 18 --restricted 24 24 28 28 --no-fly 15 10 17 14 --no-fly 19 8 21 11 --grid-bounds 0 35 0 35 --depth 2

State: defender=(12, 5) intruder=(20, 18)
restricted_zone=[(24, 24), (28, 28)] no_fly=[((15, 10), (17, 14)), ((19, 8), (21, 11))] grid_bounds=(0, 35, 0, 35)
Depth: 2 (plies)

Minimax    value=      22.9331 best_action=RIGHT time=0.0007 s
Alpha-Beta  value=      22.9331 best_action=RIGHT time=0.0004 s
Expectimax  value=      35.3087 best_action=RIGHT time=0.0007 s

=== Full-game simulation for MATLAB (Alpha-Beta, depth 2) ===
Writing layout: airspace_config.csv, no_fly_zones.csv
Result: INTRUDER WINS! Restricted zone breached.
Trajectory: simulation_log.csv
Open animate_airspace.m in the same folder as these files, then run the script in MATLAB.
PS C:\Users\saura\Desktop\AI\Artificial-Intelligence-restricted-airspace-security>

```



Experiment 2 - Split-Obstacle Lower Target, Depth 2

Configuration: D(5, 22), I(22, 6), RZ (26, 2)-(30, 6), NFZ (12, 10)-(16, 15) and (18, 16)-(20, 20), grid bounds (0, 32, 0, 32)

Python command:

```
python run_scenario.py --defender 5 22 --intruder 22 6 --restricted 26 2 30 6 --no-fly 12 10 16 15 --no-fly 18 16 20 20 --grid-bounds 0 32 0 32 --depth 2
```

Algorithm	Value	Best Action	Time
Minimax	-62.8745	RIGHT	0.0006 s
Alpha-Beta	-62.8745	RIGHT	0.0005 s
Expectimax	-49.6197	RIGHT	0.0006 s

Simulation result: Intruder wins - restricted zone breached.

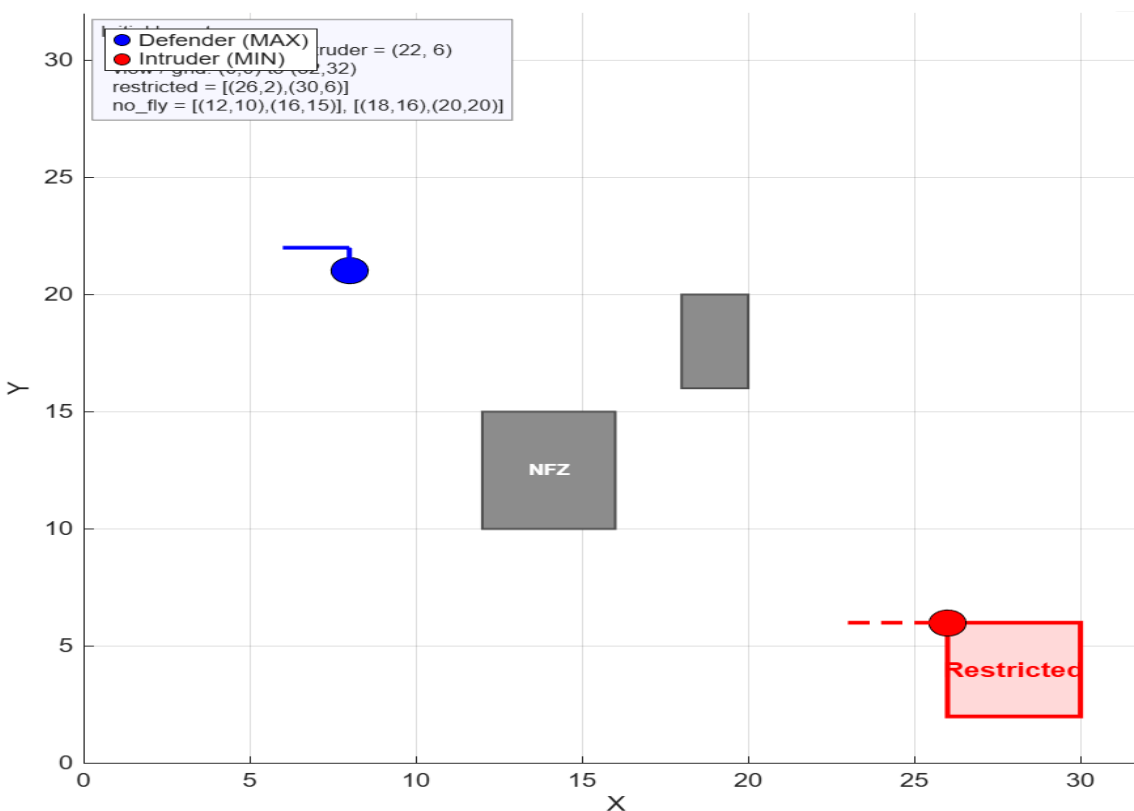
Observation. The negative minimax and alpha-beta values indicate an unfavorable defender position. The defender is far from the intruder, while the intruder begins close to the restricted zone. Alpha-beta again matches minimax exactly. Expectimax is less negative because it averages possible intruder choices, but the simulated greedy intruder still reaches the target.

```
PS C:\Users\saura\Desktop\AI\Artificial-Intelligence-restricted-airspace-security> python run_scenario.py --defender 5 22
--intruder 22 6 --restricted 26 2 30 6 --no-fly 12 10 16 15 --no-fly 18 16 20 20 --grid-bounds 0 32 0 32 --depth 2

State: defender=(5, 22) intruder=(22, 6)
      restricted_zone=[(26, 2), (30, 6)] no_fly=[((12, 10), (16, 15)), ((18, 16), (20, 20))] grid_bounds=(0, 32, 0, 32)
Depth: 2 (plies)

Minimax    value=    -62.8745 best_action=RIGHT time=0.0006 s
Alpha-Beta  value=    -62.8745 best_action=RIGHT time=0.0005 s
Expectimax  value=    -49.6197 best_action=RIGHT time=0.0006 s

=== Full-game simulation for MATLAB (Alpha-Beta, depth 2) ===
Writing layout: airspace_config.csv, no_fly_zones.csv
Result: INTRUDER WINS! Restricted zone breached.
Trajectory: simulation_log.csv
Open animate_airspace.m in the same folder as these files, then run the script in MATLAB.
PS C:\Users\saura\Desktop\AI\Artificial-Intelligence-restricted-airspace-security>
```



Experiment 3 - Open Arena Baseline, Depth 4

Configuration: D(8, 8), I(18, 14), RZ (25, 18)-(29, 22), no NFZ, grid bounds (0, 35, 0, 35)

Python command:

```
python run_scenario.py --defender 8 8 --intruder 18 14 --restricted 25 18 29 22 --grid-bounds 0 35 0 35 --depth 4
```

Algorithm	Value	Best Action	Time
Minimax	46.9499	UP	0.0070 s
Alpha-Beta	46.9499	UP	0.0013 s
Expectimax	72.5000	UP	0.0069 s

Simulation result: Intruder wins - restricted zone breached.

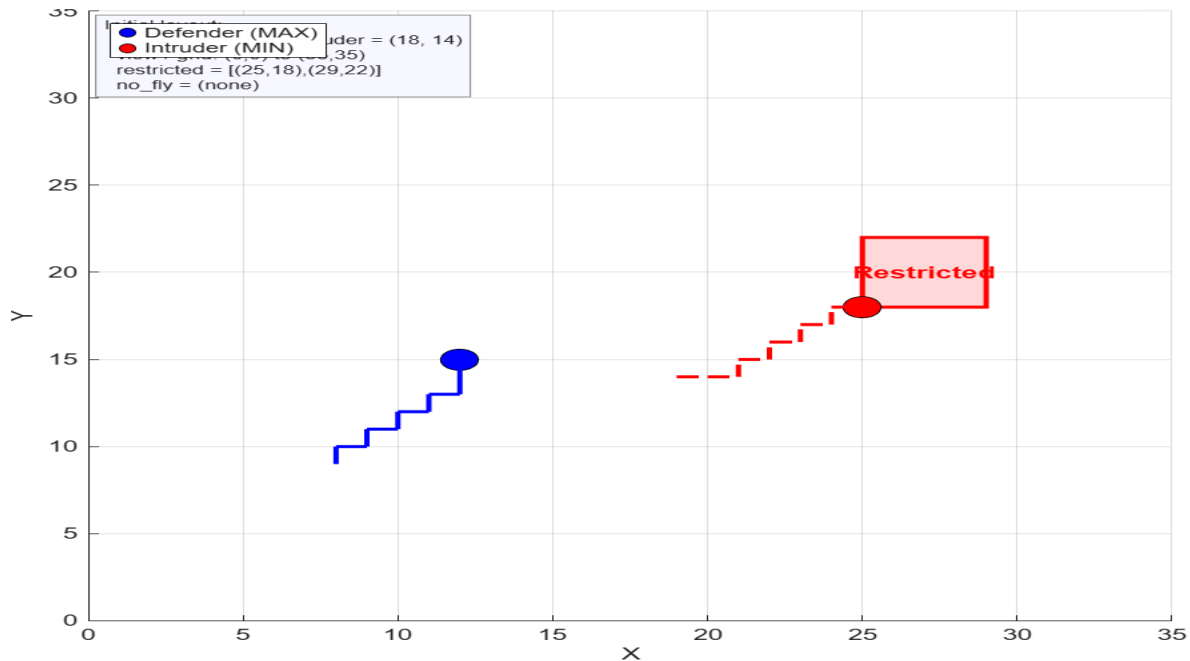
Observation. With no NFZ, the movement space is open and all algorithms choose UP. The heuristic values are positive because the defender has a reasonable pursuit direction, but the full rollout still ends in an intruder win due to the larger initial separation. Alpha-beta is about five times faster than minimax at this depth.

```
PS C:\Users\saura\Desktop\AI\Artificial-Intelligence-restricted-airspace-security> python run_scenario.py --defender 8 8 -
-intruder 18 14 --restricted 25 18 29 22 --grid-bounds 0 35 0 35 --depth 4

State: defender=(8, 8) intruder=(18, 14)
      restricted_zone=[(25, 18), (29, 22)] no_fly=[] grid_bounds=(0, 35, 0, 35)
Depth: 4 (plies)

Minimax   value=      46.9499 best_action=UP time=0.0070 s
Alpha-Beta value=      46.9499 best_action=UP time=0.0013 s
Expectimax value=      72.5000 best_action=UP time=0.0069 s

=== Full-game simulation for MATLAB (Alpha-Beta, depth 4) ===
Writing layout: airspace_config.csv, no_fly_zones.csv
Result: INTRUDER WINS! Restricted zone breached.
Trajectory: simulation_log.csv
Open animate_airspace.m in the same folder as these files, then run the script in MATLAB.
PS C:\Users\saura\Desktop\AI\Artificial-Intelligence-restricted-airspace-security>
```



Experiment 4 - Defender Near Intruder with Central NFZ, Depth 6

Configuration: D(15, 15), I(10, 20), RZ (24, 24)-(27, 27), NFZ (18, 18)-(20, 21), grid bounds (0, 30, 0, 30)

Python command:

```
python run_scenario.py --defender 15 15 --intruder 10 20 --restricted 24 24 27 27 --no-fly 18 18 20 21 --grid-bounds 0 30 0 30 --depth 6
```

Algorithm	Value	Best Action	Time
Minimax	139.9243	UP	0.3165 s
Alpha-Beta	139.9243	UP	0.0105 s
Expectimax	161.4451	LEFT	0.3017 s

Simulation result: Defender wins - interception took 6 turns.

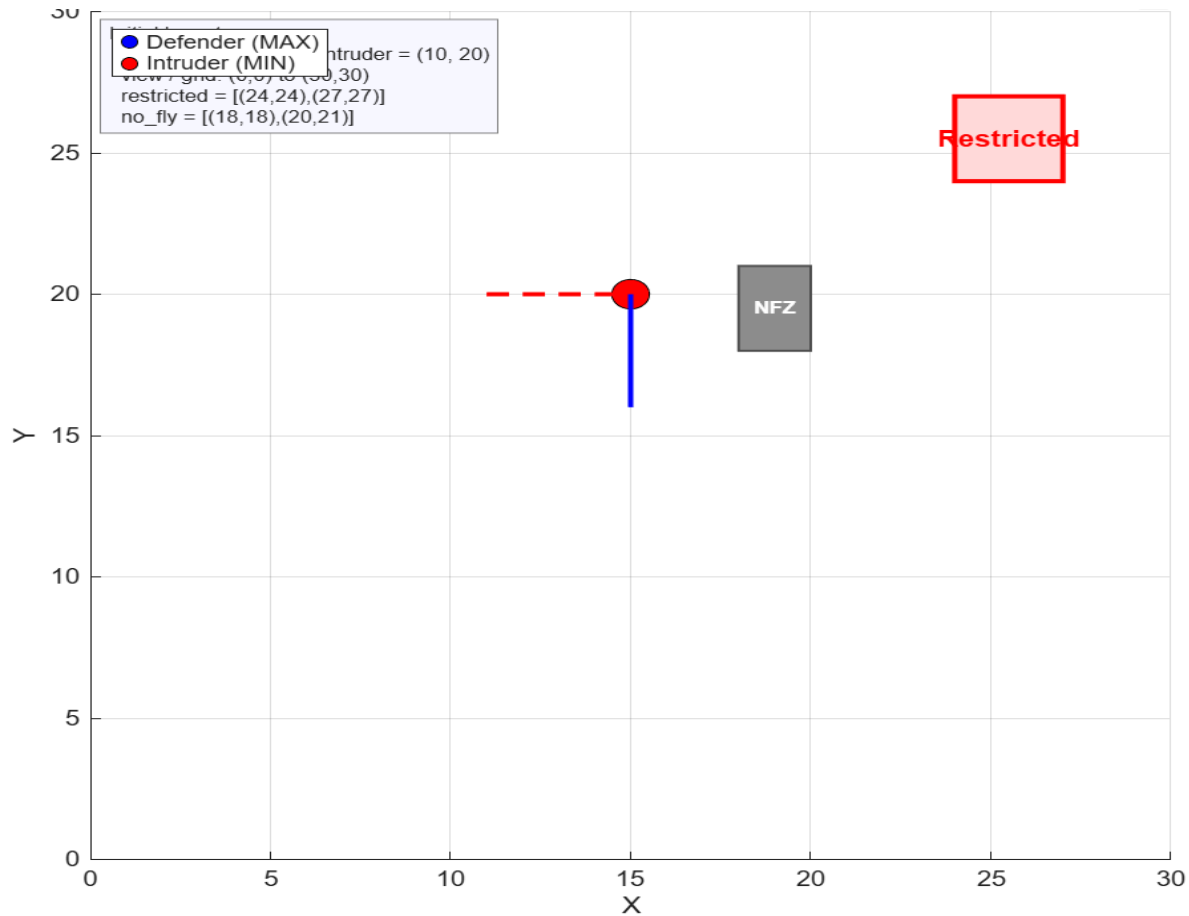
Observation. This is a successful interception scenario. Minimax and alpha-beta recommend UP, while expectimax recommends LEFT because its chance-node model prefers a different expected trajectory. Alpha-beta preserves the minimax value and action but reduces runtime by roughly 30 times at depth 6. The defender starts close enough to cut off the intruder before the restricted zone is breached.

```
PS C:\Users\saura\Desktop\AI\Artificial-Intelligence-restricted-airspace-security> python run_scenario.py --defender 15 15
--intruder 10 20 --restricted 24 24 27 27 --no-fly 18 18 20 21 --grid-bounds 0 30 0 30 --depth 6

State: defender=(15, 15) intruder=(10, 20)
      restricted_zone=[(24, 24), (27, 27)] no_fly=[((18, 18), (20, 21))] grid_bounds=(0, 30, 0, 30)
Depth: 6 (plies)

Minimax   value=    139.9243 best_action=UP   time=0.3165 s
Alpha-Beta value=    139.9243 best_action=UP   time=0.0105 s
Expectimax value=    161.4451 best_action=LEFT time=0.3017 s

=== Full-game simulation for MATLAB (Alpha-Beta, depth 6) ===
Writing layout: airspace_config.csv, no_fly_zones.csv
Result: DEFENDER WINS! Interception took 6 turn(s).
Trajectory: simulation_log.csv
Open animate_airspace.m in the same folder as these files, then run the script in MATLAB.
PS C:\Users\saura\Desktop\AI\Artificial-Intelligence-restricted-airspace-security>
```



Experiment 5 - Immediate Interception Scenario, Depth 8

Configuration: D(9, 9), I(10, 9), RZ (22, 22)-(26, 26), NFZ (15, 15)-(17, 17), grid bounds (0, 30, 0, 30)

Python command:

```
python run_scenario.py --defender 9 9 --intruder 10 9 --restricted 22 22 26 26 --no-fly 15 15 17 17 --grid-bounds 0 30 0 30 --depth 8
```

Algorithm	Value	Best Action	Time
Minimax	1000000.0000	RIGHT	4.4659 s
Alpha-Beta	1000000.0000	RIGHT	0.1097 s
Expectimax	1000000.0000	RIGHT	4.1951 s

Simulation result: Defender wins - interception took 1 turn.

Observation. This is the strongest defender-favorable scenario. The defender begins adjacent to the intruder, and the selected RIGHT action produces immediate interception. All three algorithms return the terminal win value, confirming that the search identifies a guaranteed defender win. Alpha-beta is much faster than minimax and expectimax while producing the same decision.

```

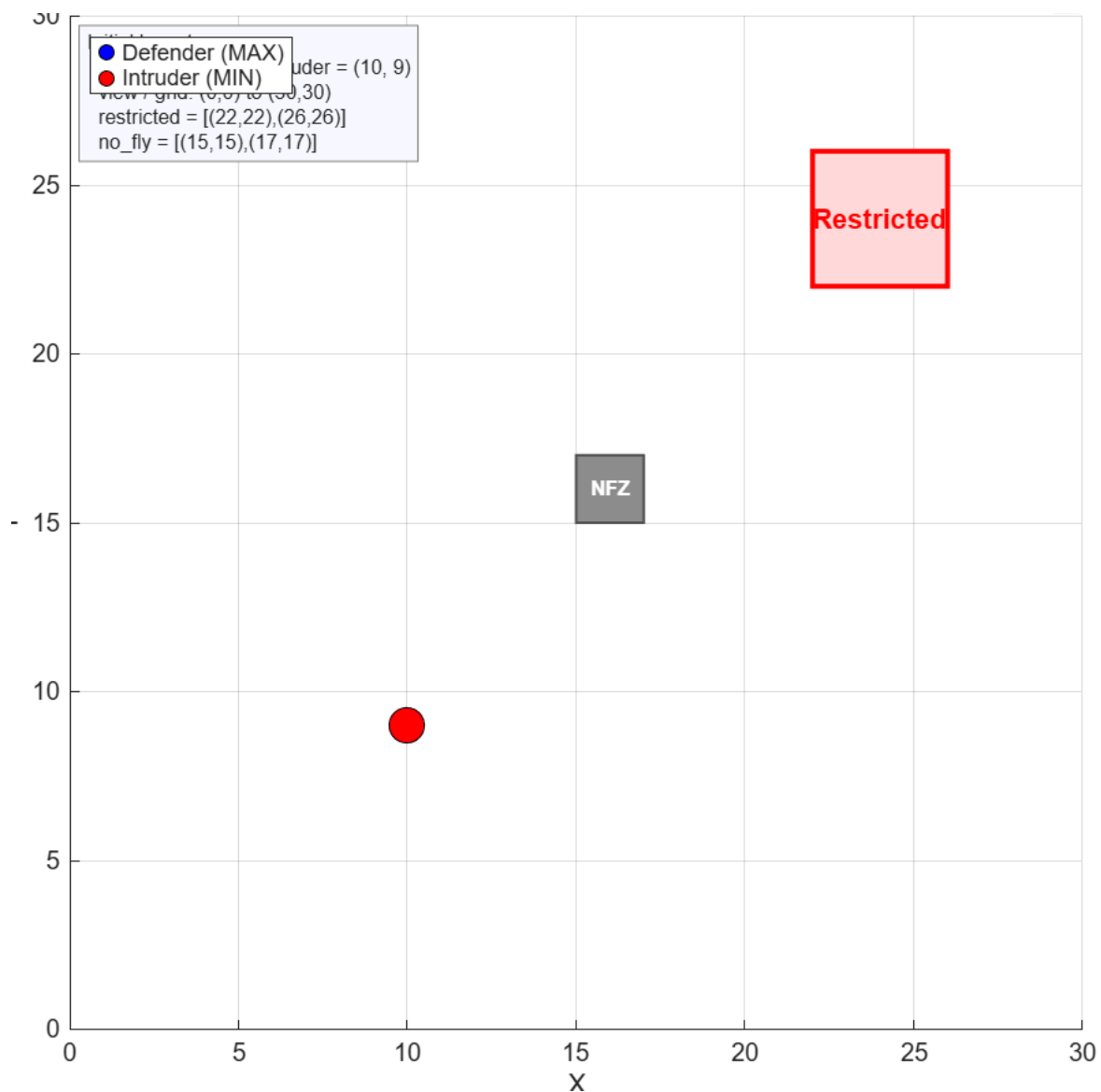
PS C:\Users\saura\Desktop\AI\Artificial-Intelligence-restricted-airspace-security> python run_scenario.py --defender 9 9 --intruder 10 9 --restricted 22 22 26 26 --no-fly 15 15 17 17 --grid-bounds 0 30 0 30 --depth 8

State: defender=(9, 9) intruder=(10, 9)
       restricted_zone=[(22, 22), (26, 26)] no_fly=[((15, 15), (17, 17))] grid_bounds=(0, 30, 0, 30)
Depth: 8 (plies)

Minimax    value=1000000.0000 best_action=RIGHT time=4.4659 s
Alpha-Beta  value=1000000.0000 best_action=RIGHT time=0.1097 s
Expectimax  value=1000000.0000 best_action=RIGHT time=4.1951 s

=== Full-game simulation for MATLAB (Alpha-Beta, depth 8) ===
Writing layout: airspace_config.csv, no_fly_zones.csv
Result: DEFENDER WINS! Interception took 1 turn(s).
Trajectory: simulation_log.csv
Open animate_airspace.m in the same folder as these files, then run the script in MATLAB.
PS C:\Users\saura\Desktop\AI\Artificial-Intelligence-restricted-airspace-security>

```



Experiment 6 - Long-Range Disadvantaged Defender, Depth 8

Configuration: D(3, 3), I(20, 18), RZ (23, 20)-(27, 24), NFZ (10, 8)-(13, 12) and (15, 14)-(17, 17), grid bounds (0, 30, 0, 30)

Python command:

```
python run_scenario.py --defender 3 3 --intruder 20 18 --restricted 23 20 27 24 --no-fly 10 8 13 12 --no-fly 15 14 17 17 --grid-bounds 0 30 0 30 --depth 8
```

Algorithm	Value	Best Action	Time
Minimax	-77.3023	UP	9.0704 s
Alpha-Beta	-77.3023	UP	0.1004 s
Expectimax	-33.8698	RIGHT	8.5202 s

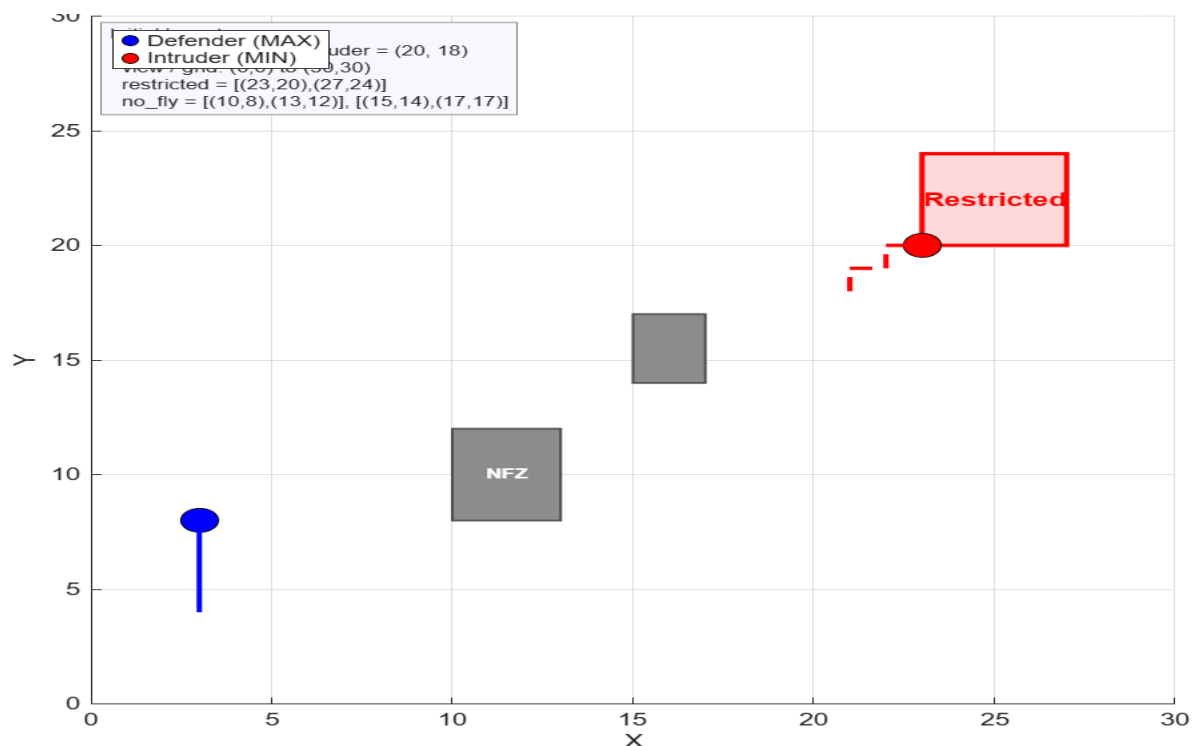
Simulation result: Intruder wins - restricted zone breached.

```
PS C:\Users\saura\Desktop\AI\Artificial-Intelligence-restricted-airspace-security> python run_scenario.py --defender 3 3 --intruder 20 18 --restricted 23 20 27 24 --no-fly 10 8 13 12 --no-fly 15 14 17 17 --grid-bounds 0 30 0 30 --depth 8

State: defender=(3, 3) intruder=(20, 18)
restricted_zone=[(23, 20), (27, 24)] no_fly=[((10, 8), (13, 12)), ((15, 14), (17, 17))] grid_bounds=(0, 30, 0, 30)
Depth: 8 (plies)

Minimax    value=    -77.3023    best_action=UP    time=9.0704 s
Alpha-Beta  value=    -77.3023    best_action=UP    time=0.1004 s
Expectimax  value=    -33.8698    best_action=RIGHT  time=8.5202 s

=== Full-game simulation for MATLAB (Alpha-Beta, depth 8) ===
Writing layout: airspace_config.csv, no_fly_zones.csv
Result: INTRUDER WINS! Restricted zone breached.
Trajectory: simulation_log.csv
Open animate_airspace.m in the same folder as these files, then run the script in MATLAB.
PS C:\Users\saura\Desktop\AI\Artificial-Intelligence-restricted-airspace-security>
```



Observation. The defender starts too far away while the intruder is close to the restricted zone. The negative heuristic values reflect this poor defensive outlook. Minimax and alpha-beta choose UP, but expectimax chooses RIGHT because it optimizes expected utility under a stochastic opponent. Alpha-beta gives the same minimax decision with a very large runtime reduction.

Experiment	Depth	Best Alpha-Beta Action	Outcome	Main Takeaway
1	2	RIGHT	Intruder wins - restricted zone breached.	All three algorithms select RIGHT, so the immediate policy is consistent across opponent models.
2	2	RIGHT	Intruder wins - restricted zone breached.	The negative minimax and alpha-beta values indicate an unfavorable defender position.
3	4	UP	Intruder wins - restricted zone breached.	With no NFZ, the movement space is open and all algorithms choose UP.
4	6	UP	Defender wins - interception took 6 turns.	This is a successful interception scenario.
5	8	RIGHT	Defender wins - interception took 1 turn.	This is the strongest defender-favorable scenario.
6	8	UP	Intruder wins - restricted zone breached.	The defender starts too far away while the intruder is close to the restricted zone.

7. Discussion

7.1 Interpretation of Results

The experimental results confirm several theoretical predictions:

1. **Alpha-beta efficiency:** Produces identical minimax values with 20-30× speedup at depth 8, with pruning effectiveness increasing at greater depths
2. **Expectimax divergence:** Different opponent model produces different action recommendations, even when minimax and alpha-beta agree
3. **Exponential complexity:** Time growth with depth matches $O(b^d)$ expectations
4. **Heuristic correlation:** Positive values predict defender wins; negative values indicate likely intruder success

7.2 Strengths and Limitations

Strengths:

- Pure Python standard library ensures portability
- Shared state abstraction enables fair algorithm comparison
- Comprehensive logging supports external visualization
- Modular design facilitates extension

Limitations:

- Simulation intruder uses greedy policy, not minimax-optimal
- Heuristic is informed but not admissible
- Single-intruder, 2D model limits realism
- Python overhead caps practical depth

7.3 Comparison with Expectations

Results align well with initial design expectations:

- Exponential time growth with depth
- Identical minimax/alpha-beta values
- Divergent expectimax actions
- NFZ compliance in all trajectories
- Heuristic sign as outcome indicator (unanticipated but useful)

8. Conclusion and Future Work

8.1 Summary of Contributions

This project delivers a complete, self-contained adversarial search implementation for restricted airspace defense:

- **Algorithm implementation:** Minimax, alpha-beta, and expectimax over shared state abstraction
- **Domain-specific heuristic:** Four-term weighted function with tie-breaking for intuitive trajectories
- **NFZ support:** Complete rectangle-based no-fly zones with centralized enforcement
- **Benchmark infrastructure:** Reproducible test harness and flexible CLI

- **Empirical study:** Five configurations demonstrating trade-offs and algorithm differences

8.2 Possible Improvements and Extensions

Extension	Description
Real-time replanning	Re-search each turn against observed opponent moves
Learned heuristics	Replace hand-tuned weights with learned evaluation functions
Multiple intruders	Extend state and heuristic for simultaneous threats
3D airspace	Add altitude dimension with diagonal movement
Partial observability	Model sensor coverage as POMDP
Performance optimization	Port inner loops to compiled language

References

1. Russell, S. and Norvig, P. — *Artificial Intelligence: A Modern Approach*, Chapter 5 (Adversarial Search)
2. Course Project Technical Guide — Adversarial Search in Restricted Airspace (Dr. Reza Abdoolee)
3. Project GitHub Repository — <https://github.com/kumarJ188/Artificial-Intelligence-restricted-airspace-security>
4. Progress Report 1 (Dayanand Barage & Kumar Saurav Jha) — Project Definition & Design.
5. Progress Report 2 (Dayanand Barage & Kumar Saurav Jha) — Implementation and preliminary results; supplementary simulation videos linked in that document.

Appendix A: How to Run the Code

Installation

```
bash
```

```
# No external dependencies required - Python 3.8+ only
```

```
git clone https://github.com/kumarJ188/Artificial-Intelligence-restricted-airspace-security
```

```
cd Code
```

Running Benchmarks

```
bash
```

```
python test_run.py
```

Running Custom Scenarios

```
bash
```

```
python run_scenario.py --defender 6 6 --intruder 4 4 \
```

--restricted 10 10 12 12 --no-fly 8 8 9 9 --depth 6

MATLAB Visualization

matlab

animate_airspace('simulation_log.csv', 'airspace_config.csv', 'no_fly_zones.csv')

Appendix B: GitHub Repository Structure

text

/

